

HTTP 완벽 가이드 · Week 2

상태 코드와 헤더

3.4 ~ 3.6 · p.67 ~ 84

발표 2-3 · 2-4 · 정우인

PART 2-3

상태 코드

3.4 · p.67 ~ 76

2xx — 요청을 정상적으로 처리했다

200

OK

조회 요청을 성공적으로
처리하고 응답

201

Created

요청이 성공해서
새로운 리소스가 생성됨

202

Accepted

요청은 접수했지만
처리는 아직 완료되지 않음

204

No Content

요청은 성공했지만
응답 본문에 보낼
데이터가 없음

정답이 정해진 게 아니라 **팀 컨벤션**에 따라 달라진다

같은 성공이라도 200으로 통일할지, 201 · 202 · 204로 세분화할지는 팀이 정한다

3xx — 리다이렉션은 크게 두 가지

301 · 308

영구적 리다이렉션

“이 주소는 이제 안 써. 앞으로 새 주소로 와”

- ✓ 브라우저가 **캐싱** → 옛 주소를 입력해도 서버에 묻지 않고 새 주소로 요청
- ✓ 검색엔진도 **새 주소로 교체**

예시 도메인 변경, http → https

302 · 303 · 307

일시적 리다이렉션

“지금만 잠깐 저기로 가. 원래 주소는 살아 있어”

- ✓ 브라우저가 **기억하지 않음** → 매번 원래 주소로 먼저 요청
- ✓ 검색엔진도 **원래 주소를 유지**

예시 로그인 안 한 사용자를 로그인 페이지로 보낼 때

차이는 “브라우저와 검색엔진이 새 주소를 기억하느냐” — 코드별 세부 차이는 다음 장에서

301 vs 308 — 메서드와 본문을 유지하는가

301 Moved Permanently

1
요청

```
POST /event HTTP/1.1
Host: localhost:8080
name=hello&age=20
```

2
응답

```
HTTP/1.1 301 Moved Permanently
Location: /new-event
```

3
자동
재요청

```
GET /new-event HTTP/1.1
Host: localhost:8080
(본문 사라짐)
```

✗ POST → GET 으로 바뀌고 본문 제거

308 Permanent Redirect

1
요청

```
POST /event HTTP/1.1
Host: localhost:8080
name=hello&age=20
```

2
응답

```
HTTP/1.1 308 Permanent Redirect
Location: /new-event
```

3
자동
재요청

```
POST /new-event HTTP/1.1
Host: localhost:8080
name=hello&age=20
```

✓ POST 와 본문을 그대로 유지

POST 요청을 301로 리다이렉트하면 폼 데이터가 사라진다 → 본문을 지켜야 하면 308

302 · 307 · 303 — 리다이렉트 후 메서드는?

리소스의 URL이 **일시적으로** 바뀐 것 · 검색엔진 등은 URL을 바꾸면 안 된다

302

Found

...될 수도, 안 될 수도 있다

메서드 **GET으로 변경 ?**

본문 **제거**

구현마다 동작이 제각각

307

Temporary Redirect

302와 기능은 같음

메서드 **유지**

본문 **유지**

메서드와 본문을 그대로

303

See Other

302와 기능은 같음

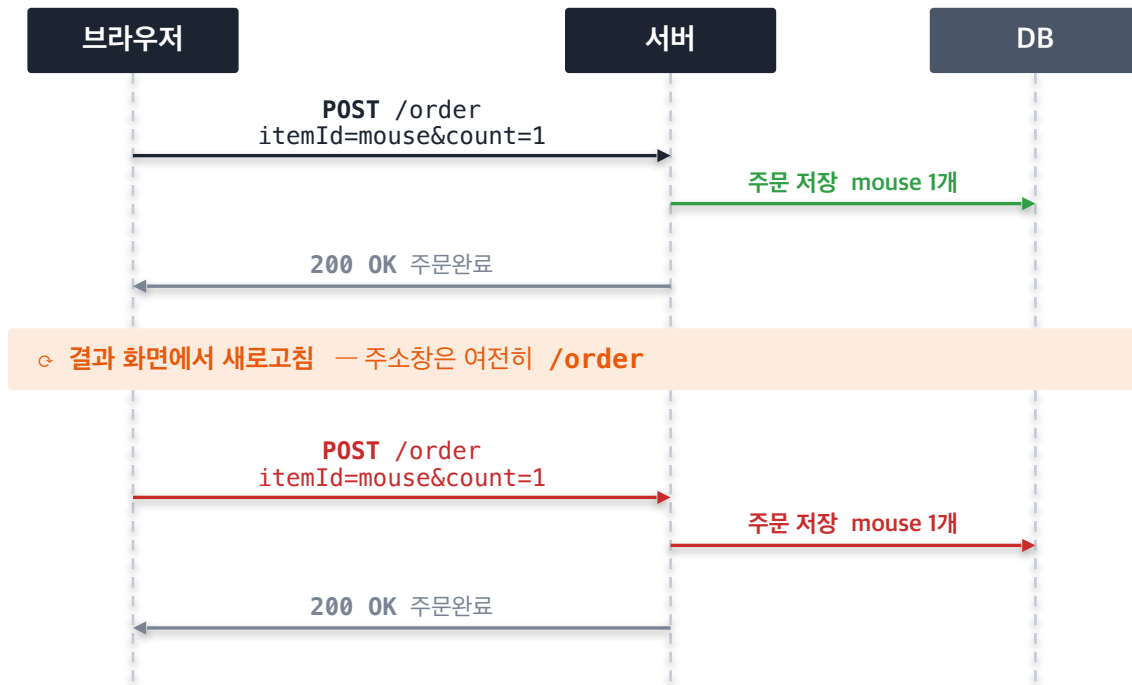
메서드 **GET으로 변경**

본문 **제거**

반드시 GET으로 바뀌서

애매한 302를 둘로 나눔 → **유지하려면 307** · **GET으로 바꾸려면 303**

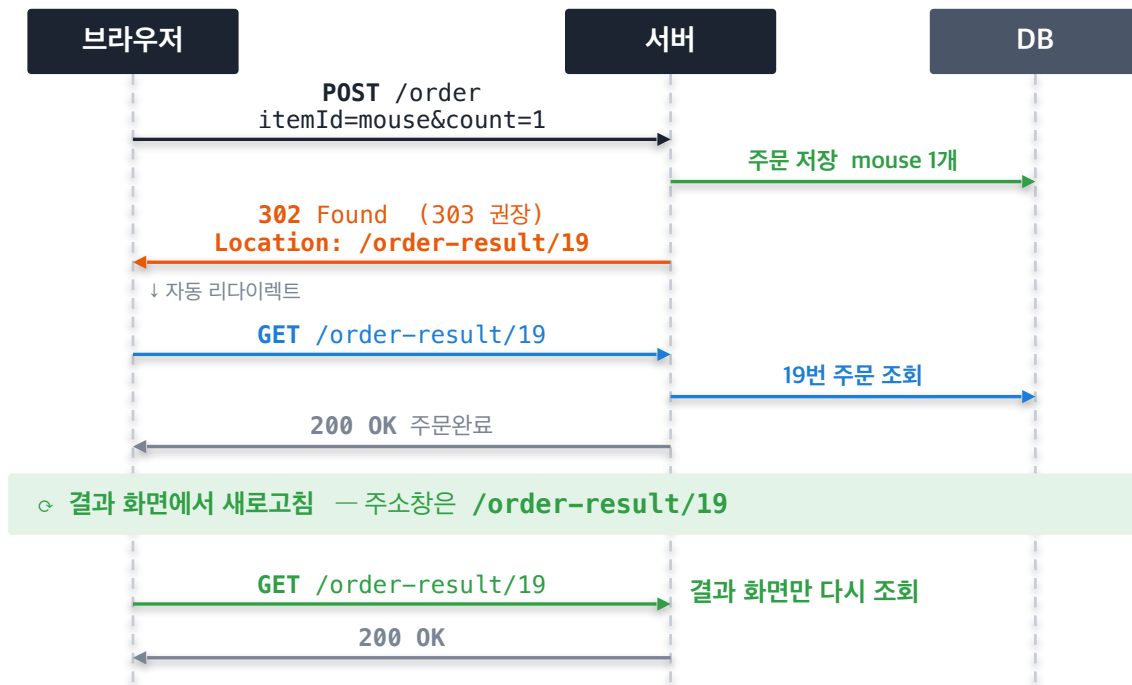
PRG 사용 전 — 새로고침하면 주문이 두 번



무슨 일이 일어났나

- 1 POST 결과를 200으로 바로 응답
- 2 새로고침 = 마지막 요청 다시 전송
- 3 같은 POST가 또 가서 주문이 중복 저장

PRG 사용 후 — Post / Redirect / Get



핵심

- 1 POST로 주문한 뒤
결과 화면으로 리다이렉트
- 2 결과 화면은 GET으로
조회 → 새로고침해도 안전

최종 주문 응답은
200 이 아니라
302 (303 권장)

304 — 첫 요청: 캐시에 저장할 정보를 받는다

1 첫 요청

```
GET /star.jpg HTTP/1.1
Host: localhost:8080
```

2 첫 응답

```
HTTP/1.1 200 OK
Content-Type: image/jpeg
Content-Length: 34012
Cache-Control: max-age=60
ETag: "abc123"
Last-Modified: Tue, 29 Sep 2026 10:00:00 GMT
(이미지 데이터 34KB)
```

Cache-Control: max-age=60

60초 동안은 서버에 묻지 말고
캐시를 그냥 써도 된다

ETag

이 파일의 버전 이름표

Last-Modified

마지막 수정 시간

브라우저는 이미지 + 이 정보들을 캐시에 저장

304 — 60초 뒤: “아직 그대로야?”

3 60초 지난 후 재요청 — 가진 정보를 같이 보낸다

```
GET /star.jpg HTTP/1.1
Host: localhost:8080
If-None-Match: "abc123"
If-Modified-Since: Tue, 29 Sep 2026 10:00:00 GMT
```

“나 abc123 버전 갖고 있는데,
아직 그대로야?”

4-a 안 바뀌었을 때

```
HTTP/1.1 304 Not Modified
Cache-Control: max-age=60
ETag: "abc123"
(바디 없음)
```

바디가 없어 응답이 **아주 가볍다**
캐시 이미지를 쓰고 유효시간을 다시 60초로 갱신

4-b 바뀌었을 때

```
HTTP/1.1 200 OK
ETag: "xyz789"
Last-Modified: Wed, 30 Sep 2026 09:00:00 GMT
(새 이미지 데이터)
```

새 파일과 새 ETag를 받아서
캐시를 교체

참고 둘 다 오면 ETag(If-None-Match)가 우선 · 수정 시간은 초 단위라 1초 안의 변경을 못 잡고, 내용이 같아도 시간만 바뀌면 “바뀜”으로 본다

4xx — 클라이언트 쪽 오류

400 Bad Request

잘못된 요청이라 서버가 처리할 수 없다

요청 파라미터 · API 스펙 불일치, 구문 오류

→ 클라이언트가 요청을 검토해서 다시 보내야

서버는 스펙을 철저히 검증해 400 반환 500으로 넘기지 않기

401 Unauthorized

인증 ×

인증이 필요한데, 인증되지 않았다

해당 리소스에 접근하려면 인증이 필요

응답에 **WWW-Authenticate** 헤더로

인증 방법을 함께 알려준다

403 Forbidden

권한 ×

요청은 이해했지만 승인을 거부

인증 자격은 있지만 접근 권한이 불충분

예) 일반 사용자로 로그인한 뒤

어드민 등급 리소스에 접근

404 Not Found

요청한 리소스를 찾을 수 없다

요청한 리소스가 서버에 없다

또는 권한이 부족한 클라이언트에게

리소스의 존재 자체를 숨기고 싶을 때

5xx — 서버 쪽 오류

서버는 500 에러를 **함부로 만들면 안 된다**

- ✓ 서버가 터지거나 오류가 있을 때 → 진짜 서버에 문제가 있을 때만 500
- ✗ 비즈니스 로직 · 비즈니스 조건의 문제 → 500을 쓰면 안 된다

500

Internal Server Error

서버 문제로 오류 발생

애매하면 500

503

Service Unavailable — 서비스 이용 불가

- 일시적인 과부하 나 예정된 작업으로 잠시 요청을 처리할 수 없음
- **Retry-After** 헤더로 언제 복구되는지 알려줄 수도 있음
- 서버 오류는 대부분 예측 불가 → 503을 보는 경우는 흔치 않음

PART 2-4

헤더

3.5 ~ 3.6 · p.76 ~ 84

헤더 — HTTP 전송에 필요한 부가정보

시작줄

HTTP/1.1 200 OK

헤더

Content-Type: text/html;charset=UTF-8
Content-Length: 3423
Cache-Control: max-age=60
Date: Thu, 01 Oct 2026 10:00:00 GMT

빈 줄

CRLF

본문

```
<html>
  <body>...</body>
</html>
```

형식

Host

:

www.google.com

필드명

필드값

필드명은 대소문자 구분 없음

역할 HTTP 전송에 필요한 모든 부가정보

본문 타입 Content-Type

압축 Content-Encoding

인증 Authorization

클라이언트 정보 User-Agent

캐시 관리 Cache-Control

서버 정보 Server

헤더는 다섯 가지로 분류한다

일반 헤더	클라이언트와 서버 양쪽 모두에서 사용	Date	Connection	Cache-Control
요청 헤더	요청 메시지를 위한 헤더	Host	User-Agent	If-None-Match
응답 헤더	응답 메시지를 위한 헤더	Server	WWW-Authenticate	Retry-After
엔터티 헤더	엔터티 본문에 대한 헤더	Content-Type	Content-Length	ETag
확장 헤더	애플리케이션 개발자들이 만든 헤더 명세에는 추가되지 않음	X-Request-ID	X-Forwarded-For	

오른쪽은 예시 헤더 — 앞에서 본 If-None-Match(304), WWW-Authenticate(401), Retry-After(503)도 여기에 속한다

지금 짚고 갈 요청 · 응답 헤더

3xx 와 짝

응답

Location

리다이렉트할 새 주소

Location: /order-result/19

401 과 짝

응답 ↔ 요청

WWW-Authenticate ↔ Authorization

인증 방법을 알려주면
클라이언트가 인증 정보를 담아 재요청

Authorization: Bearer eyJ...

503 과 짝

응답

Retry-After

얼마 뒤에 다시 요청하면 되는지
(초 또는 날짜)

Retry-After: 120

HTTP/1.1 필수

요청

Host

요청한 호스트 도메인 — 한 IP에
여러 도메인이 있어도 구분(가상 호스팅)

Host: www.google.com

클라이언트 정보

요청

User-Agent

어떤 브라우저 · 앱이 보냈는지
통계, 특정 브라우저 장애 파악

User-Agent: Mozilla/5.0 ...

유입 경로

요청

Referer

직전에 보던 페이지 주소
(철자는 명세의 오타가 그대로 굳은 것)

Referer: https://www.google.com/

나머지 헤더는 각 주차에서 다시 만난다

Week 3~4	커넥션	Connection	Keep-Alive	Upgrade			
Week 5	엔터티와 인코딩	Content-Type	Content-Length	Content-Encoding	Transfer-Encoding	Range	
Week 6~7	캐시	Cache-Control	Expires	Age	ETag	Last-Modified	If-None-Match
Week 8	협상	Accept	Accept-Language	Accept-Encoding	Content-Language	Vary	
Week 9	쿠키	Set-Cookie	Cookie				
Week 10	HTTPS	Strict-Transport-Security					

오늘은 분류와 상태 코드에 붙는 헤더까지 — 나머지는 그 주차 발표자에게 바통을 넘긴다